

UstaadAssist

Your learning plan: every backend file, in the order that makes sense, with the exact guide pages to read alongside it.

Use it together with UstaadAssist-Backend-Guide.pdf
Nine stages, 52 files, one tracker
29 September 2026

Contents

Every chapter and section, with the page it starts on. In a PDF viewer the same list is in the bookmarks panel, and each entry is clickable.

1 · How to use this plan	3
Why a plan, and not just reading from the top	3
The two documents, and how they fit together	3
The loop — do this for every file	3
Tools that make learning faster	5
How long it takes	5
2 · The stages	6
Stage 0 — Foundations: how the project runs	6
Stage 1 — The front door: a request's journey	6
Stage 2 — The database	8
Stage 3 — Who is asking	8
Stage 4 — Your own code first, then its neighbours	9
Stage 5 — The planner (the heart of the project)	10
Stage 6 — The daily work: classes, attendance, students, marks	11
Stage 7 — Reading photos and PDFs	11
Stage 8 — Around the app: scripts and deployment	12
The final check	12
3 · Progress tracker	13
Stage 0 — Foundations	13
Stage 1 — The front door	13
Stage 2 — The database	13
Stage 3 — Who is asking	13
Stage 4 — Your own code, then its neighbours	14
Stage 5 — The planner	14
Stage 6 — The daily work	15
Stage 7 — Reading photos and PDFs	15
Stage 8 — Around the app	15
Words I did not know	16

1 · How to use this plan

The goal: know this backend like the palm of your hand. Not "I have seen every file" — "I could explain every line, and rebuild it if I had to".

Why a plan, and not just reading from the top

If you open the project and scroll, every file looks equally unfamiliar, because each file leans on things from other files. `plan.service.ts` uses `transaction()`, which is in `pool.ts`, which uses `env`, which is in `env.ts`. Reading it in the wrong order means meeting ten unknown words in every file.

So this plan puts the files in an order where **each stage only uses things you learned in the stages before it**. Nothing in Stage 3 depends on Stage 5. By the time you reach the planner, every piece of plumbing it touches is already familiar, and you can spend all your attention on the algorithm.

The two documents, and how they fit together

Document	What it is for
This plan	The route: which file, in what order, what to do with it, and how you know you are done
UstaadAssist-Backend-Guide.pdf	The explanations: what each file does and why, line by line, from zero

Every stage below tells you which **guide pages** to read with it. The guide's page numbers are at the bottom right of every page, and its bookmarks panel (the sidebar in any PDF viewer) jumps straight to any section. The index at the back of the guide lists every file and function with its page.

The loop — do this for every file

- 1. Read the file once, top to bottom, without stopping.** Do not look anything up yet. Just get the shape: how long is it, what are the functions called, what does it import.
- 2. Write down every line you do not understand.** Copy it exactly. Do not be embarrassed by how many there are — the first few files will have many, and that number drops fast.
- 3. Read the guide pages for that file.** Most of your lines will be answered there.
- 4. Bring me whatever is still unclear.** Paste the lines and say which file they are from. I will explain each one starting from what you already know, and define every new word as it comes up.
- 5. Explain the file back, in your own words, in 3–5 sentences.** Write it or say it out loud to me. This is the step that actually makes it stick: you do not really understand something until you can explain it without looking. I will point out what you missed.
- 6. Break something on purpose.** Change one line, run it, and watch what happens. Every stage below suggests an experiment. Afterwards, undo it:

```
git checkout src/app.ts      # puts the file back exactly as it was
```

7. Write one line in your notebook: "file does **this**. It is used by **that**." By the end, those lines are a map of the whole project in your own handwriting.

The rule that matters most: **do not move on while a line is still a mystery**. One skipped line becomes three confusing lines in the next file.

Tools that make learning faster

Running one test file

The tests are the best examples in the project. Each one calls a real function with real input and checks the output. To run just one file:

```
node --experimental-strip-types --test tests/planner.test.ts
```

Read a test, predict what it checks, then run it.

The VS Code debugger — watching code run line by line

This is the single best tool for understanding the planner.

1. In VS Code, open the terminal menu (the + dropdown) and choose **JavaScript Debug Terminal**. 2. Open `src/planner/slots.ts` and click in the margin to the left of a line number. A red dot appears — that is a **breakpoint**: "pause here". 3. In the debug terminal, run a test file (the command above). 4. The code stops at your red dot. Hover over any variable to see its value. Press **F10** to run one line, **F11** to step into a function.

Watching `cursor` move one day at a time through the semester teaches `generateSlots` better than any explanation.

Calling the server yourself

Start it:

```
npm run dev
```

Then, in another terminal:

```
curl http://localhost:4000/health
```

Most endpoints need a token. `scripts/smoke.sh` shows how to get one: it signs in as the demo teacher (`demo.teacher@ustaadassist.test`) through Supabase and keeps the `access_token`. Copy that `curl` line from the top of the script to get your own token, then send it:

```
curl http://localhost:4000/courses -H "Authorization: Bearer <paste token>"
```

`console.log` — the oldest tool

Add `console.log("here", someVariable);` anywhere, save (`npm run dev` restarts by itself), call the endpoint, and read the terminal. Remove it afterwards.

How long it takes

Roughly one stage every day or two. The planner stage (Stage 5) deserves three or four days on its own — it is the heart of the project. There is no prize for speed: a stage is done when its "done when" boxes are all ticked, not when a day has passed.

2 · The stages

Nine stages, in order. Each one lists the files, the guide pages to read with them, one experiment, and how to know you are done.

Stage 0 — Foundations: how the project runs

Before any code: what happens when you type `npm run dev`?

File	What it is	Guide
<code>package.json</code>	The project's name, its packages, and the <code>npm run</code> commands	pages 9–10
<code>tsconfig.json</code>	Settings for the TypeScript type checker	not in the guide — ask me
<code>.env.example</code>	The list of secret settings the server needs	page 10

Also read: chapter 1 (pages 6–14) for the big picture, and chapter 2 (pages 15–21) for the TypeScript and JavaScript syntax used everywhere. Chapter 2 is a reference — skim it now, come back to it whenever a symbol like `??` or `!` confuses you.

Experiment: open `package.json`, find the `dev` script, and work out what each part of `node --experimental-strip-types --watch --env-file=.env src/index.ts` does.

Done when

- ☐ I can say what `npm run dev` actually runs, piece by piece
- ☐ I know what `.env` is for and why it is never committed
- ☐ I can name the five runtime packages and what each is for

Stage 1 — The front door: a request's journey

How a request arrives, which files it passes through, and how the answer goes back.

File	What it does	Guide
<code>src/index.ts</code>	Starts the app on a port	page 51
<code>src/app.ts</code>	Builds the app: the order every request travels in	pages 50–51
<code>src/env.ts</code>	Refuses to start if a setting is missing	pages 49–50
<code>src/http.ts</code>	<code>ok()</code> , <code>AppError</code> , <code>pathId()</code> — the response shape	pages 42–44
<code>src/middleware/error.ts</code>	The only place an error answer is sent	pages 44–46

Also read: "The whole request flow" (page 7), and "Two Express-specific things that will bite" (page 21).

Experiments:

- Move `app.use(notFoundHandler);` above the routers in `app.ts`. Call `/health`. Why does every URL now answer 404?
- Delete the `_next` parameter from `errorHandler`. Call a URL that throws. What happens, and why?
- In `.env`, rename `DATABASE_URL` to `DATABASE_URLX` and start the server. Read the message `env.ts` prints.

(Undo each one with `git checkout <file>.`)

Done when

- ☐ I can draw a request's path from the phone to the reply on paper, without looking
- ☐ I can explain why the order of lines in `app.ts` matters
- ☐ I can explain every line of `app.ts`
- ☐ I know why a route never sends an error itself

Stage 2 — The database

What is stored, and how the code talks to it.

File	What it does	Guide
migrations/001_init.sql	Creates 15 tables	pages 22–27
migrations/002_makeup_date.sql	Adds the 16th table	page 22
migrations/003_security.sql	Locks the tables against direct access	pages 27–28
src/db/pool.ts	The connection pool and <code>transaction()</code>	pages 46–49

Also read: "The SQL techniques used, from zero" (pages 28–33). Every SQL trick in the project is explained there: `$1`, `unnest`, `on conflict`, `count(*) filter`, `coalesce`, `to_char`.

Experiments:

- In the Supabase SQL editor, run `select * from session limit 10;` and match the columns with the `create table session` lines in `001_init.sql`.
- Try to insert a topic with `min_sessions = 5` and `sessions_needed = 2`. Read the error. Which constraint stopped it?

Done when

- ☐ I can draw all 16 tables and the arrows (foreign keys) between them
- ☐ I can explain why `mark.obtained` has no default of 0
- ☐ I can explain why `transaction()` borrows one connection and holds it
- ☐ I can explain what Row Level Security does here and why the server is not locked out

Stage 3 — Who is asking

How the server knows which teacher sent a request, and how it stops one teacher reading another's data.

File	What it does	Guide
src/middleware/requireAuth.ts	Checks the Supabase token	pages 34–40
src/routes/auth.ts	<code>GET /auth/me</code> , which creates the teacher row	page 40
src/services/ownership.ts	"Does this teacher own this row?"	pages 40–41

Experiments:

- Call `GET /courses` with no token, then with a token that has one character changed. Compare the two answers.
- Decode your token at `jwt.io` (paste it in; it is only your demo account) and find the `sub` field. Where does that value end up in the database?

Done when

- ☐ I can explain how the server checks a token **without** asking Supabase each time

- ☐ I know what `sub` is and where it is used
- ☐ I can explain why somebody else's course answers 404, not 403

Stage 4 — Your own code first, then its neighbours

You wrote the first part of `courses.ts` yourself. Start there, on familiar ground, then read the code written next to it.

File	What it does	Guide
<code>src/routes/courses.ts</code> (the first ~200 lines)	Create a course, save topics, holidays	pages 91–93
<code>src/data/pakHolidays.ts</code>	The public holidays that get preloaded	pages 14, 92
<code>src/services/course.service.ts</code>	Course settings, topic edits, cloning	pages 87–88
<code>src/routes/topics.ts</code>	PATCH <code>/topics/:id</code>	page 92

Notice: your routes put the SQL right inside the route. The later code moved SQL into **services** and kept the routes to one line each. Read both and work out what the service style gains (hint: the seed script calls services directly, page 111).

Experiment: create a course with `curl` (the body is on page 91), then fetch its holidays. Where did they come from?

Done when

- ☐ I can explain every line of my own code in `courses.ts`, including the `unnest` queries
- ☐ I can explain how `updateCourse` builds its SQL from only the fields that were sent
- ☐ I can explain what `cloneCourse` copies and what it deliberately does not

Stage 5 — The planner (the heart of the project)

Take your time here. This is the part that makes the app more than a data-entry system.

Read each planner file **next to its tests**. The test file shows you what the function is supposed to do; the source shows you how.

File	What it does	Guide
<code>src/planner/slots.ts</code>	Which dates can class happen on	pages 52–56
<code>src/planner/allocate.ts</code>	Spreads topics over those dates	pages 56–58
<code>src/planner/assessments.ts</code>	Moves a quiz that falls before its topics (binary search)	pages 58–60
<code>src/planner/health.ts</code>	How many weeks behind the teacher is	page 69
<code>src/planner/replan.ts</code>	Freezes the past, rebuilds the future	pages 60–64
<code>src/planner/deficit.ts</code>	Three ways out of a shortage (knapsack)	pages 64–69
<code>tests/planner.test.ts</code> , <code>tests/replan.test.ts</code>	49 tests that pin the planner down	pages 69–70
<code>src/services/plan.service.ts</code>	Load, think, save	pages 78–81
<code>src/routes/plan.ts</code>	The planner endpoints	pages 93–95

Read in this order: slots → allocate → assessments → health → replan → deficit. Each one uses the ones before it.

Experiments — do these on paper first, then check with the code:

- **Slots.** A course from Mon 7 Sep to Fri 25 Sep 2026, teaching Mon and Wed, with 16 Sep a holiday. List every slot with its week number. Then put a breakpoint in `generateSlots` and watch it produce the same list.
- **Allocation.** Three topics needing 1, 2 and 1 classes on those slots. Which topic lands on which date?
- **Binary search.** Using the worked example on page 59, find the first slot after 13 Sep by hand, step by step.
- **Knapsack.** Short by 3 classes, low-priority topics of 2, 2 and 3 classes. Fill in the `best` table on paper, the way pages 64–67 describe, and walk it back. Then run the test `drop finds one topic where greedy would have dropped two`.
- **Replan.** In a test, cancel one class and read what `changes` reports. Why is it "moved" and not "added"?

Done when

- ☐ I can work out a small plan by hand and get the same answer as the code
- ☐ I can explain "freeze the past, rebuild the future", and point at the exact line that does the freezing
- ☐ I can explain why greedy is not enough for Drop, and how the table fixes it
- ☐ I can explain binary search with the worked example, without notes
- ☐ I can explain why the planner never touches the database

Stage 6 — The daily work: classes, attendance, students, marks

Now that sessions exist (Stage 5), everything else hangs off them.

File	What it does	Guide
<code>src/services/session.service.ts + src/routes/sessions.ts</code>	Mark a class conducted or cancelled	pages 81–82, 96
<code>src/services/attendance.service.ts</code>	Present-by-default attendance	pages 82–84, 96
<code>src/services/student.service.ts + src/routes/students.ts</code>	Save a reviewed class list	pages 84–85, 96–97
<code>src/grading/compute.ts + tests/grading.test.ts</code>	Weighted totals and grades — pure	pages 71–77
<code>src/services/grading.service.ts + src/routes/assessments.ts</code>	Assessments, marks, weightage	pages 85–87, 97–99
<code>src/services/material.service.ts + src/routes/materials.ts</code>	Course material records	pages 99–101
<code>src/services/dashboard.service.ts</code>	Every dashboard number at once	pages 88–89
<code>src/services/report.service.ts</code>	The three reports (and the PDF stub)	pages 89, 108–110

Experiments:

- Mark a class conducted with `PATCH /sessions/:id`, then check the topic's `status`. Who changed it, and how?
- Take one student from the seeded course. Compute their weighted total by hand from their marks and the weightage, then compare with `GET /courses/:id/results`.

Done when

- ☐ I can explain why attendance only sends the absentees
- ☐ I can explain the three grading rules: round once, missing is not zero, absent is zero
- ☐ I can explain why the result screen and the result PDF can never disagree

Stage 7 — Reading photos and PDFs

File	What it does	Guide
<code>src/services/extraction.service.ts</code>	Downloads the upload, as the teacher	pages 102–103
<code>src/extraction/readText.ts</code>	PDF text, or OCR on a photo	pages 103–105
<code>src/extraction/classList.ts</code>	Finds seat numbers and names; the vote	pages 105–106
<code>src/extraction/outline.ts</code>	Finds topics in a course outline	pages 106–107
<code>tests/extraction.test.ts</code>	12 tests, no photos needed	page 107

Experiment: in `tests/extraction.test.ts`, find the sample text of the class list. Change one seat number's `0` to a letter `o` and run the tests. What does the parser do with it, and what confidence does the row get?

Done when

- ☐ I can explain why the review screen is not optional
- ☐ I can read the seat-number regular expression piece by piece
- ☐ I can explain how six OCR readings vote on one name

Stage 8 — Around the app: scripts and deployment

File	What it does	Guide
<code>scripts/create-demo-user.ts</code>	Creates the demo login	pages 112–113
<code>scripts/seed.ts</code>	Builds a full demo semester	pages 111–112
<code>scripts/smoke.sh</code>	Calls every endpoint and checks the answer	pages 113–115
<code>api/index.ts</code>	Hands the app to Vercel	pages 116–117
<code>vercel.json</code>	Sends every URL to that one function	page 119

Experiment: run `npm run seed`, then `npm run smoke` with the course id it prints. Read two of the smoke checks that are **supposed** to fail, and explain why failing is correct.

Done when

- ☐ I can explain why the seed script calls services instead of inserting rows
- ☐ I can explain the difference between `src/index.ts` and `api/index.ts`

The final check

Read chapter 14 of the guide (pages 121–123). Cover the answers to "The five questions most likely to be asked" and answer each one out loud. If you can answer all five, and tick every box in this plan, you know this backend.

3 · Progress tracker

Every file in the backend, in the order of the plan. Tick the three columns as you go: read it, explained it back, and broke something in it on purpose.

Stage 0 — Foundations

File	Read	Explained	Experimented
package.json	☐	☐	☐
tsconfig.json	☐	☐	☐
.env.example	☐	☐	☐

Stage 1 — The front door

File	Read	Explained	Experimented
src/index.ts	☐	☐	☐
src/app.ts	☐	☐	☐
src/env.ts	☐	☐	☐
src/http.ts	☐	☐	☐
src/middleware/error.ts	☐	☐	☐

Stage 2 — The database

File	Read	Explained	Experimented
migrations/001_init.sql	☐	☐	☐
migrations/002_makeup_date.sql	☐	☐	☐
migrations/003_security.sql	☐	☐	☐
src/db/pool.ts	☐	☐	☐

Stage 3 — Who is asking

File	Read	Explained	Experimented
src/middleware/requireAuth.ts	☐	☐	☐
src/routes/auth.ts	☐	☐	☐
src/services/ownership.ts	☐	☐	☐

Stage 4 — Your own code, then its neighbours

File	Read	Explained	Experimented
src/routes/courses.ts	☐	☐	☐
src/data/pakHolidays.ts	☐	☐	☐
src/services/course.service.ts	☐	☐	☐
src/routes/topics.ts	☐	☐	☐

Stage 5 — The planner

File	Read	Explained	Experimented
src/planner/slots.ts	☐	☐	☐
src/planner/allocate.ts	☐	☐	☐
src/planner/assessments.ts	☐	☐	☐
src/planner/health.ts	☐	☐	☐
src/planner/replan.ts	☐	☐	☐
src/planner/deficit.ts	☐	☐	☐
tests/planner.test.ts	☐	☐	☐
tests/replan.test.ts	☐	☐	☐
src/services/plan.service.ts	☐	☐	☐
src/routes/plan.ts	☐	☐	☐

Stage 6 — The daily work

File	Read	Explained	Experimented
src/services/session.service.ts	☐	☐	☐
src/routes/sessions.ts	☐	☐	☐
src/services/attendance.service.ts	☐	☐	☐
src/services/student.service.ts	☐	☐	☐
src/routes/students.ts	☐	☐	☐
src/grading/compute.ts	☐	☐	☐
tests/grading.test.ts	☐	☐	☐
src/services/grading.service.ts	☐	☐	☐
src/routes/assessments.ts	☐	☐	☐
src/services/material.service.ts	☐	☐	☐
src/routes/materials.ts	☐	☐	☐
src/services/dashboard.service.ts	☐	☐	☐
src/services/report.service.ts	☐	☐	☐

Stage 7 — Reading photos and PDFs

File	Read	Explained	Experimented
src/services/extraction.service.ts	☐	☐	☐
src/extraction/readText.ts	☐	☐	☐
src/extraction/classList.ts	☐	☐	☐
src/extraction/outline.ts	☐	☐	☐
tests/extraction.test.ts	☐	☐	☐

Stage 8 — Around the app

File	Read	Explained	Experimented
scripts/create-demo-user.ts	☐	☐	☐
scripts/seed.ts	☐	☐	☐
scripts/smoke.sh	☐	☐	☐
api/index.ts	☐	☐	☐
vercel.json	☐	☐	☐

Words I did not know

Every time you meet a word you had to look up, write it here with a one-line meaning in your own words. By the end this is your personal glossary.

Word	What it means, in my words